

AI-Assisted Writing Detection — SQL Analysis

Likhitha V | MySQL Workbench | Dataset: cleaned_ai_writing_dataset (10,000 rows)

Phase 2 of 3 — Python (EDA) → SQL (Querying) → Power BI (Dashboard)

Easy

Q1 Find the total number of submissions in the dataset.

```
SELECT COUNT(*) AS total_submission
FROM cleaned_ai_writing_dataset;
```

Q2 List the first 10 submissions where Is_AI_Assisted = 1.

```
SELECT *
FROM cleaned_ai_writing_dataset
WHERE Is_AI_Assisted = 1
LIMIT 10;
```

Q3 Find all submissions with a Plagiarism_Score greater than 20.

```
SELECT *
FROM cleaned_ai_writing_dataset
WHERE Plagiarism_Score > 20
LIMIT 10;
```

Q4 Get the distinct values in Academic_Level and Submission_Type.

```
SELECT DISTINCT Academic_Level
FROM cleaned_ai_writing_dataset;

SELECT DISTINCT Submission_Type
FROM cleaned_ai_writing_dataset;
```

Q5 Find the top 5 submissions with the highest Word_Count, sorted descending.

```
SELECT *
FROM cleaned_ai_writing_dataset
ORDER BY Word_Count DESC
LIMIT 5;
```

Medium

Q6 Find the average Typing_Speed and Revision_Count for each value of Is_AI_Assisted.

```
SELECT Is_AI_Assisted,
       AVG(Typing_Speed) AS avg_typing_speed,
       AVG(Revision_Count) AS avg_revision_count
FROM cleaned_ai_writing_dataset
GROUP BY Is_AI_Assisted;
```

Q7 Count how many submissions fall into each Academic_Level, broken down by Is_AI_Assisted.

```
SELECT Academic_Level, Is_AI_Assisted, COUNT(*) AS total
FROM cleaned_ai_writing_dataset
GROUP BY Academic_Level, Is_AI_Assisted
ORDER BY Academic_Level;
```

Q8 Find the average Word_Count per Submission_Type.

```
SELECT Submission_Type, ROUND(AVG(Word_Count), 2) AS avg_word_count
FROM cleaned_ai_writing_dataset
GROUP BY Submission_Type;
```

Q9 Find the maximum and minimum Grammar_Errors for AI-assisted vs non-AI-assisted submissions.

```
SELECT Is_AI_Assisted,
       MAX(Grammar_Errors) AS max_grammar_errors,
       MIN(Grammar_Errors) AS min_grammar_errors
FROM cleaned_ai_writing_dataset
GROUP BY Is_AI_Assisted;
```

Q10 Find how many submissions were made in each Submission_Hour, ordered by hour.

```
SELECT Submission_Hour, COUNT(*) AS total_submissions
FROM cleaned_ai_writing_dataset
GROUP BY Submission_Hour
ORDER BY Submission_Hour;
```

Hard

Q11 Find academic levels where the average Plagiarism_Score is above the overall average Plagiarism_Score (subquery).

```
SELECT Academic_Level, ROUND(AVG(Plagiarism_Score), 2) AS avg_plagiarism_score
FROM cleaned_ai_writing_dataset
GROUP BY Academic_Level
HAVING AVG(Plagiarism_Score) > (
    SELECT AVG(Plagiarism_Score)
    FROM cleaned_ai_writing_dataset
);
```

Q12 Using CASE, label Submission_Hour into Morning / Afternoon / Evening / Night buckets, then count each.

```
SELECT
    CASE
        WHEN Submission_Hour BETWEEN 6 AND 11 THEN 'Morning'
        WHEN Submission_Hour BETWEEN 12 AND 17 THEN 'Afternoon'
        WHEN Submission_Hour BETWEEN 18 AND 21 THEN 'Evening'
        ELSE 'Night'
    END AS Time_Bucket,
    COUNT(*) AS Submission_Count
FROM cleaned_ai_writing_dataset
GROUP BY Time_Bucket;
```

Q13 Find the top 3 Submission_Type categories with the highest proportion (%) of AI-assisted submissions.

```
SELECT Submission_Type,
       AVG(Is_AI_Assisted) * 100 AS ai_assisted_percentage
FROM cleaned_ai_writing_dataset
GROUP BY Submission_Type
ORDER BY ai_assisted_percentage DESC
LIMIT 3;
```

Q14 Using HAVING, find Academic_Level + Submission_Type combinations where AI-assisted count > 50.

```
SELECT Academic_Level, Submission_Type, COUNT(*) AS total_ai_assisted
FROM cleaned_ai_writing_dataset
WHERE Is_AI_Assisted = 1
GROUP BY Academic_Level, Submission_Type
HAVING COUNT(*) > 50;
```

Q15 Using a window function, rank submissions within each Academic_Level by Word_Count and return the top 3 per level.

```
SELECT *
FROM (
    SELECT Academic_Level,
           Word_Count,
           RANK() OVER (PARTITION BY Academic_Level ORDER BY Word_Count DESC) AS word_rank
    FROM cleaned_ai_writing_dataset
) AS ranked
WHERE word_rank <= 3;
```

